

TECNOLOGIA EM DESENVOLVIMENTO DE SOFTWARE MULTIPLATAFORMA

TÉCNICA DE PROGRAMAÇÃO I
REVISÃO



Técnica de Programação I

Profº Luiz Cláudio

Fatec Registro CPS SÃO PAULO

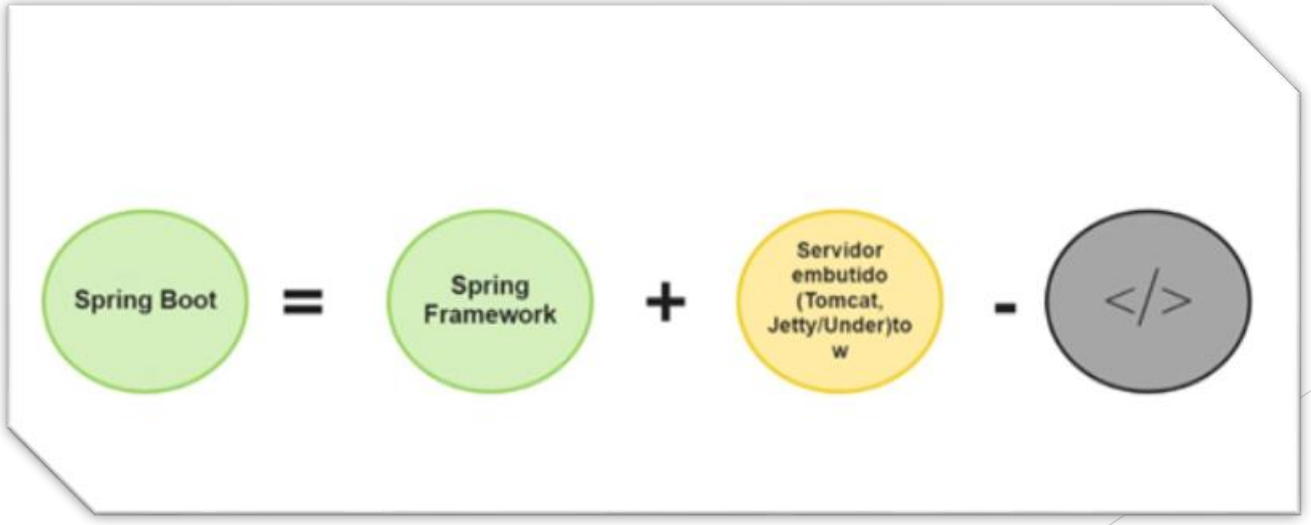

spring
Boot



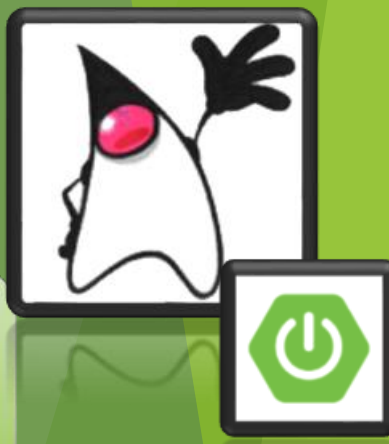
O que é Spring Boot?



O **Spring Boot** é um framework **Open Source**, que nasceu a partir do Spring framework e veio para facilitar as configurações iniciais de um projeto.



Conceitos Básicos Spring Boot



Injeção de Dependências: O Spring Boot gerencia automaticamente as dependências entre os componentes da sua aplicação, eliminando a necessidade de configurá-las manualmente.

Camadas de Abstração: O framework oferece diversas camadas de abstração que simplificam o desenvolvimento de tarefas comuns, como acesso a banco de dados, segurança e integração com serviços externos.

Anotações Poderosas: As anotações do Spring Boot são a chave para configurar e personalizar sua aplicação de forma intuitiva e eficiente.

Spring Initializr: Utilize essa ferramenta online para gerar projetos Spring Boot pré-configurados, agilizando ainda mais o processo de desenvolvimento.

Conceitos Básicos Spring Boot

Spring Boot é um framework que facilita a criação de aplicações Java, especialmente aplicações web e APIs REST. (Representational State Transfer) é uma interface que permite a comunicação entre sistemas através do protocolo HTTP

Vantagens do Spring Boot:



_Configuração automática
_Servidor embutido (Tomcat)
_Integração com Spring MVC(Model-View-Controller)
_JPA(**Java Persistence API) ->** facilitando a gravação, leitura e manipulação de objetos Java em bancos de dados
_Produtividade

Conceitos Básicos Spring Boot

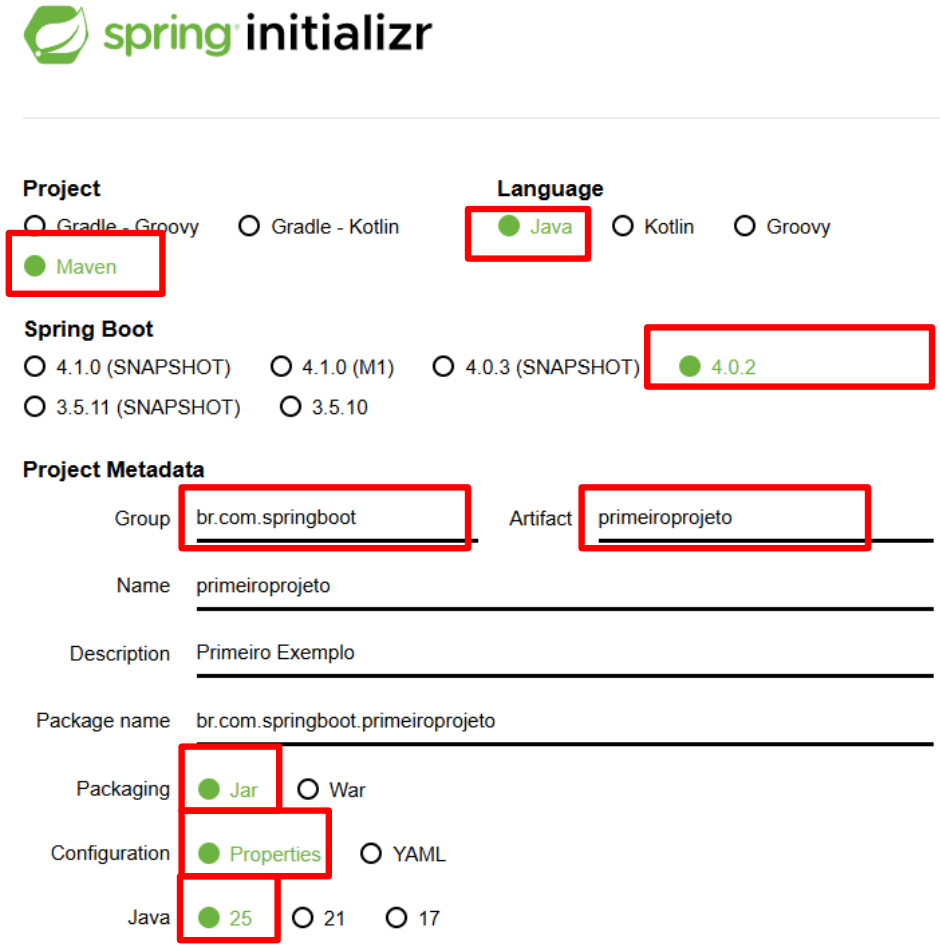
Estrutura básica de um projeto Spring Boot:

- controller: recebe as requisições HTTP
- repository: acesso ao banco de dados
- entity: entidades do sistema

PRIMEIRO PROJETO COM SPRING BOOT

Utilizaremos o Spring Initializr para criar a estrutura inicial do projeto.

Acesse o site https://start.spring.io/ , para criar o projeto com spring boot



Selecione em **Project** a opção Maven

Language Java

Em **Spring Boot** – mantenha a versão instalada no seu computador, nesta caso 4.0.2

Em **Group**, defina o pacote de armazenamento do projeto, Normalmente inciamos com br.com.SEGUIDODONOME, neste caso springboot

Em **Artififact** , defina o nome do seu projeto , o mesmo deve ser colocado em Name

Em **Description**, não é obrigatório mas pode preencher com descrição do seu projeto

Em **Package Name** , ficara a junção do pacote com nome do projeto

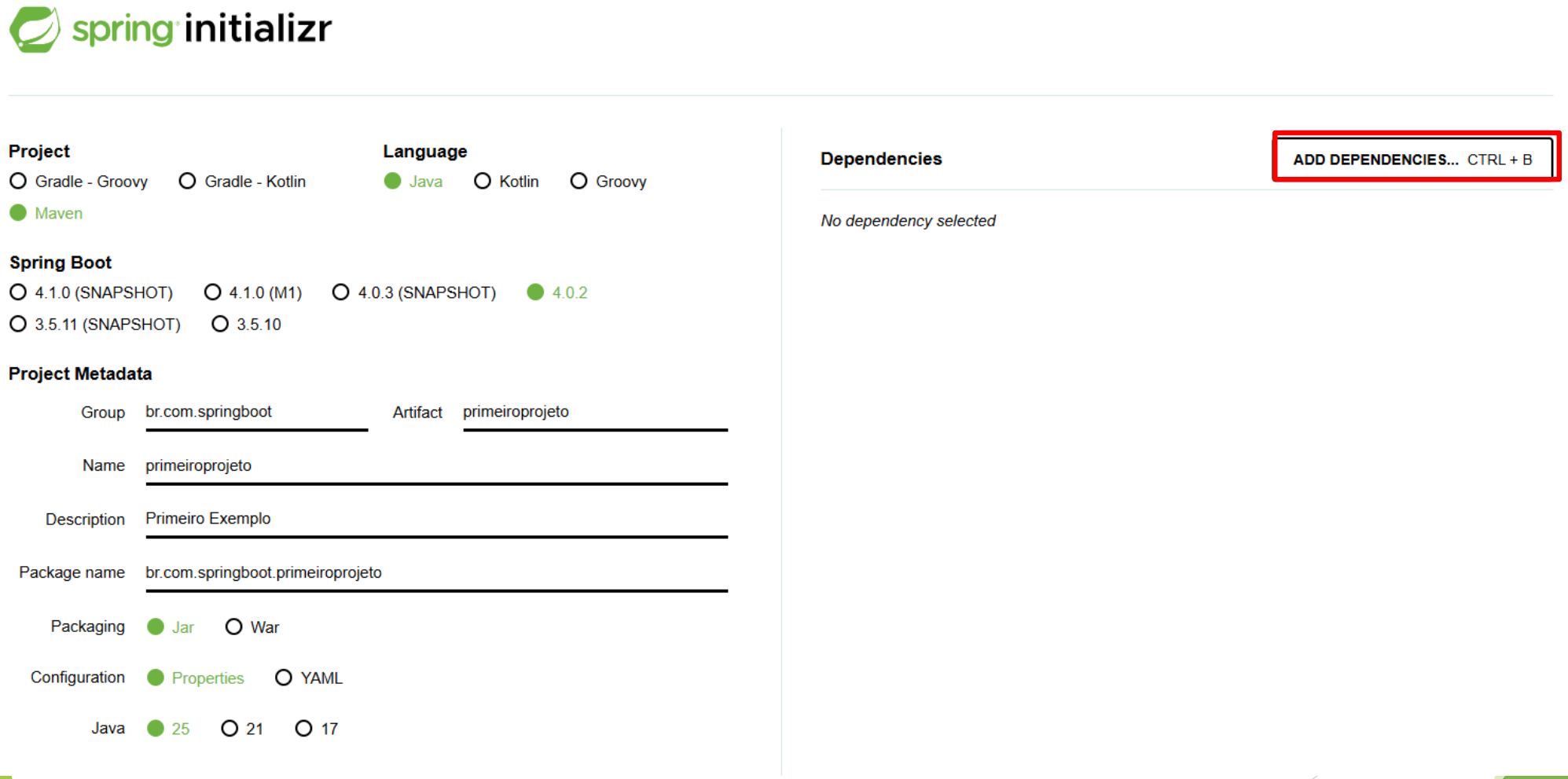
Em **Packing** , mantenha o grupo de pacote Jar

Em **Configuration** , mantenha properties

Em **Java** , deixa a versão instalada no seu computador

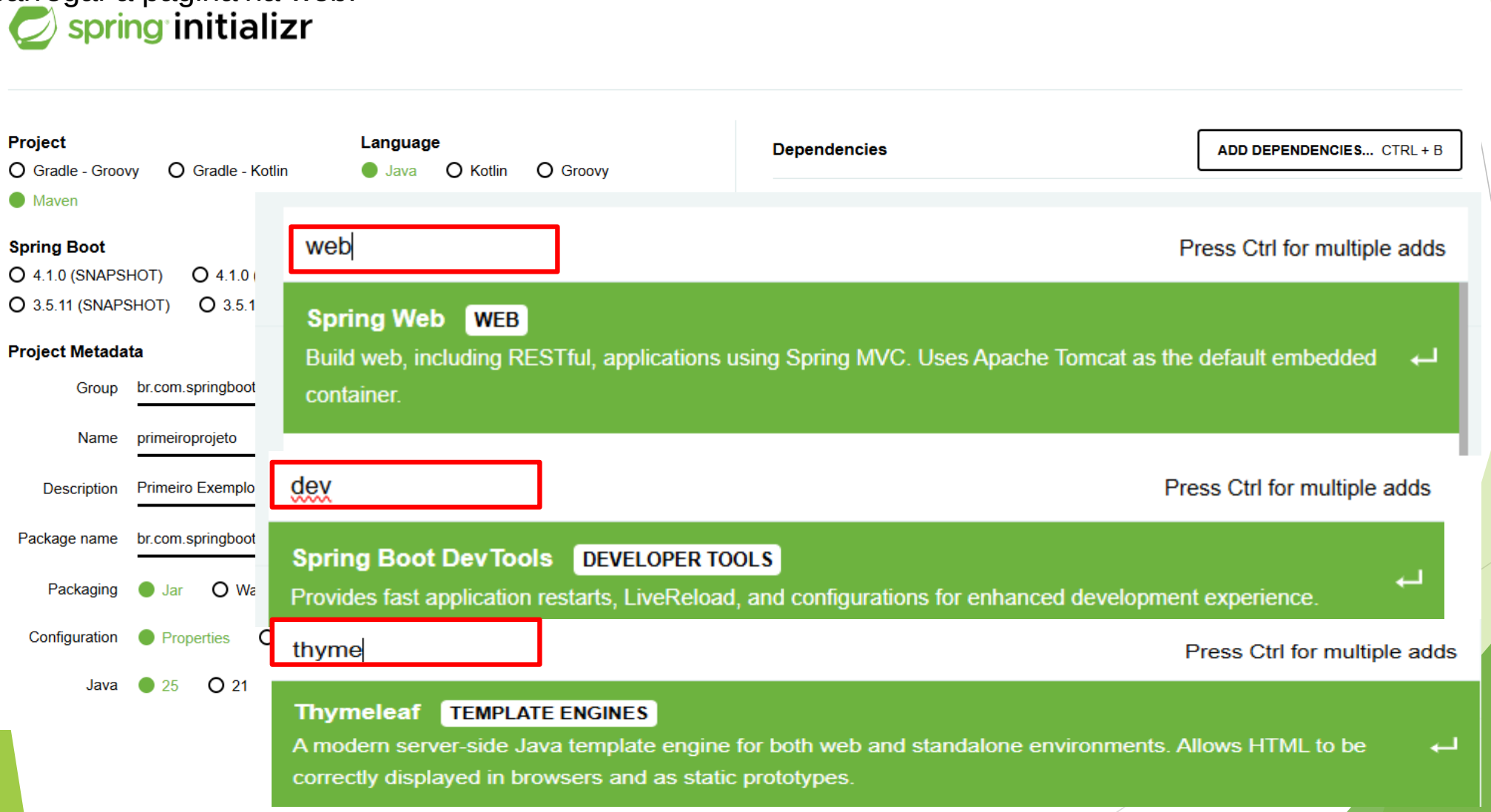
PRIMEIRO PROJETO COM SPRING BOOT

Em seguida clique em add Dependencies...



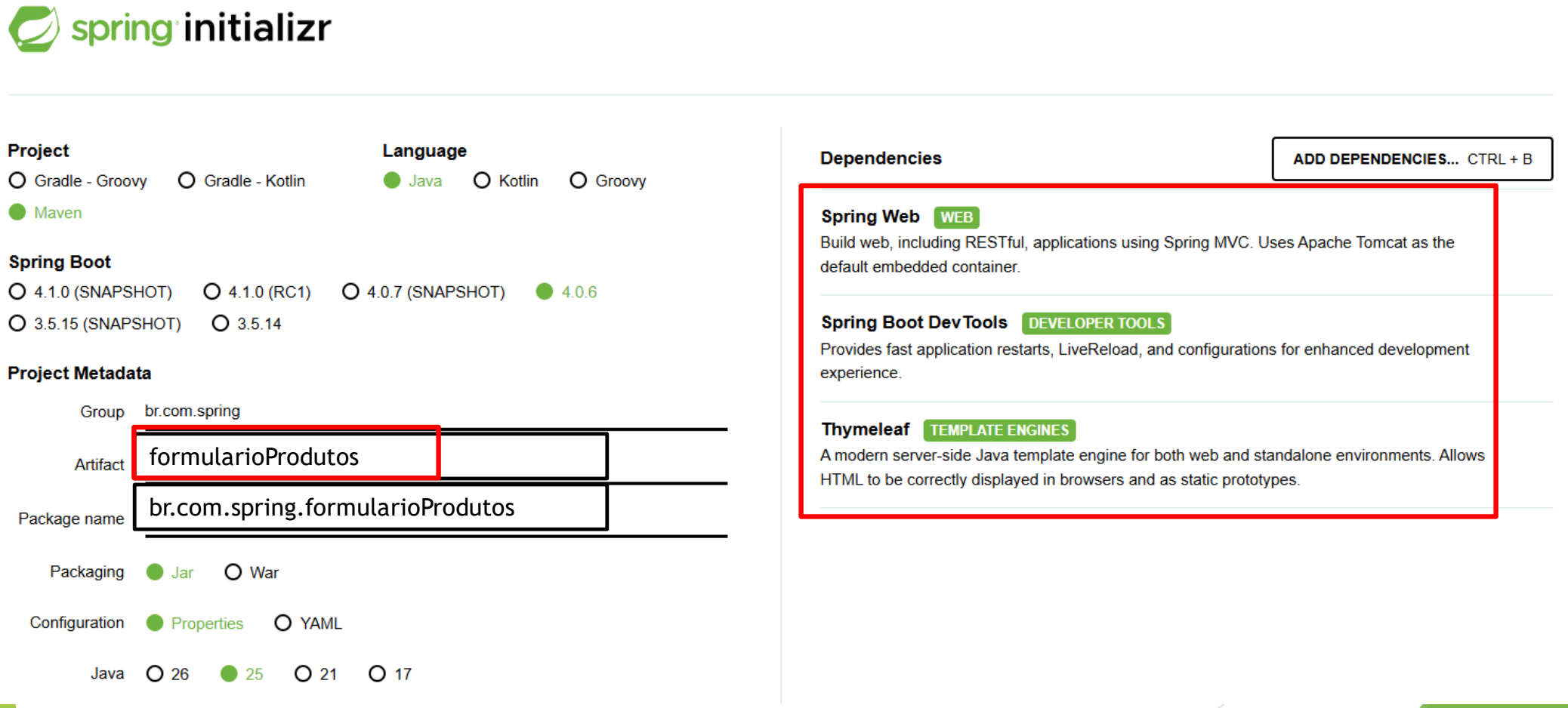
PRIMEIRO PROJETO COM SPRING BOOT

Digite nas dependências web e selecione Spring Web, para rodar o projeto na web , em seguida digite dev , e selecione o spring boot devtools , para que o projeto atualize automaticamente, digite thyme, para adicionar o thymeleaf e ter acesso e carregar a página na web.



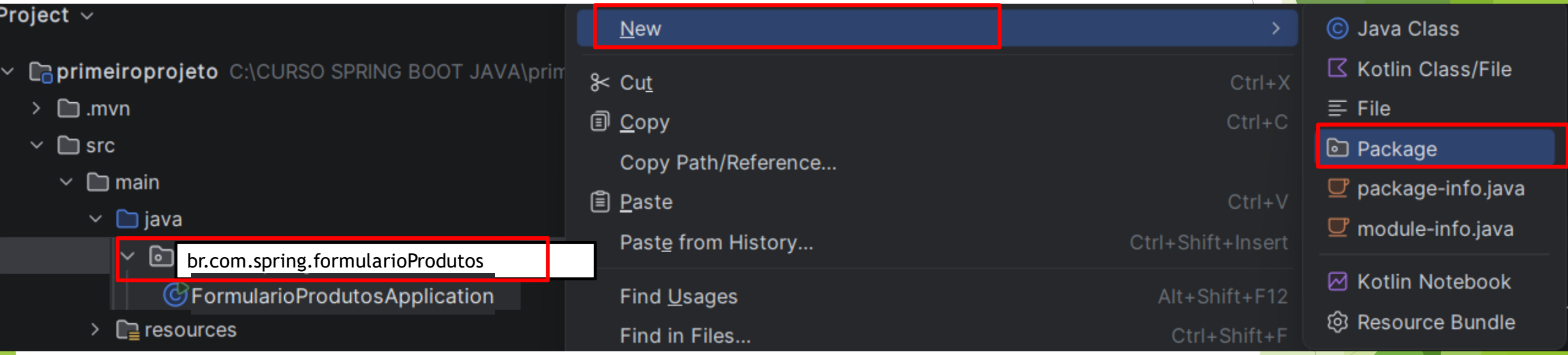
PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

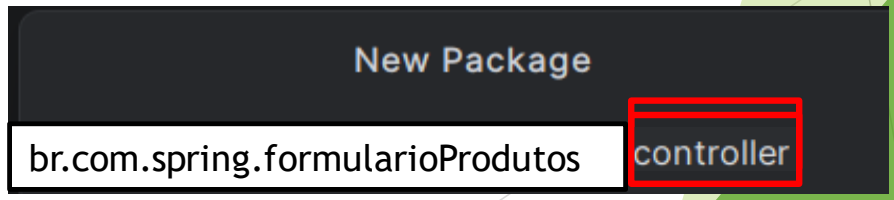
Entre no site https://start.spring.io/, para criar o projeto spring boot.Depois de adicionar as dependências(web,dev,thyme) , clique em generate , para criar e baixar seu projeto



PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

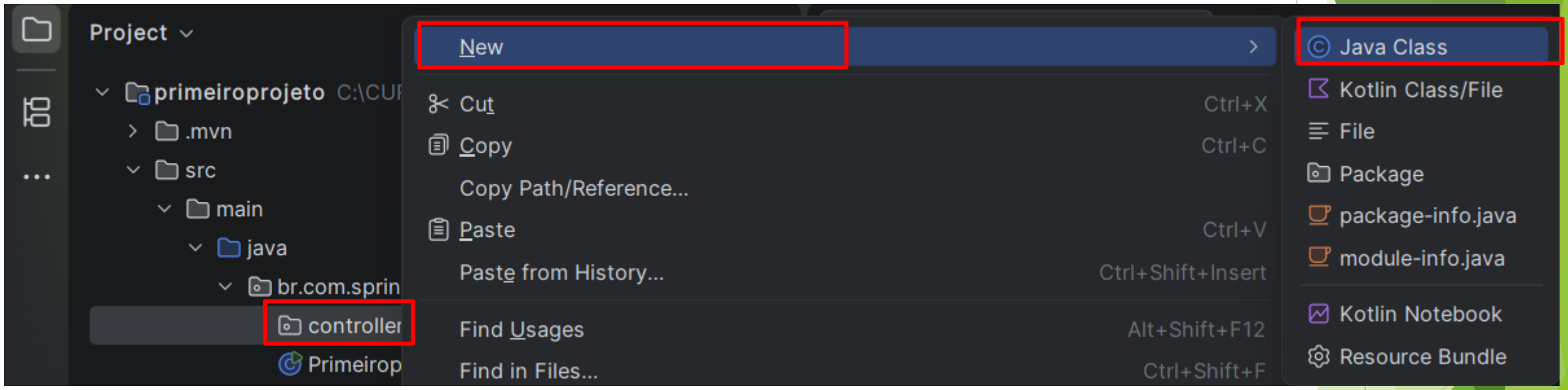
Para iniciarmos , vamos criar um pacote controller , pressione o botão direito , sobre o nome do pacote do projeto , selecione New->Package e digite controller

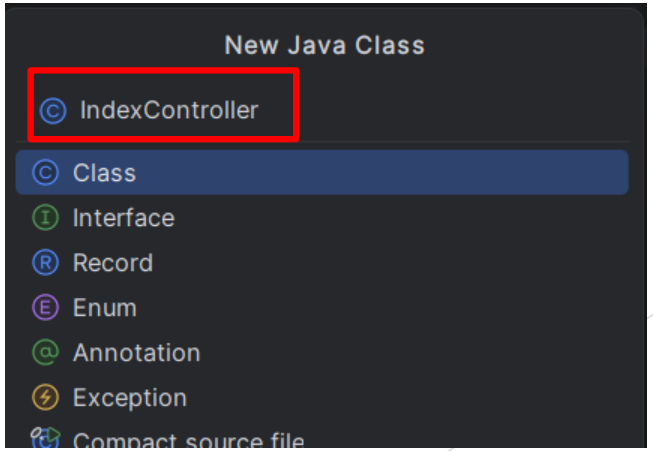




PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

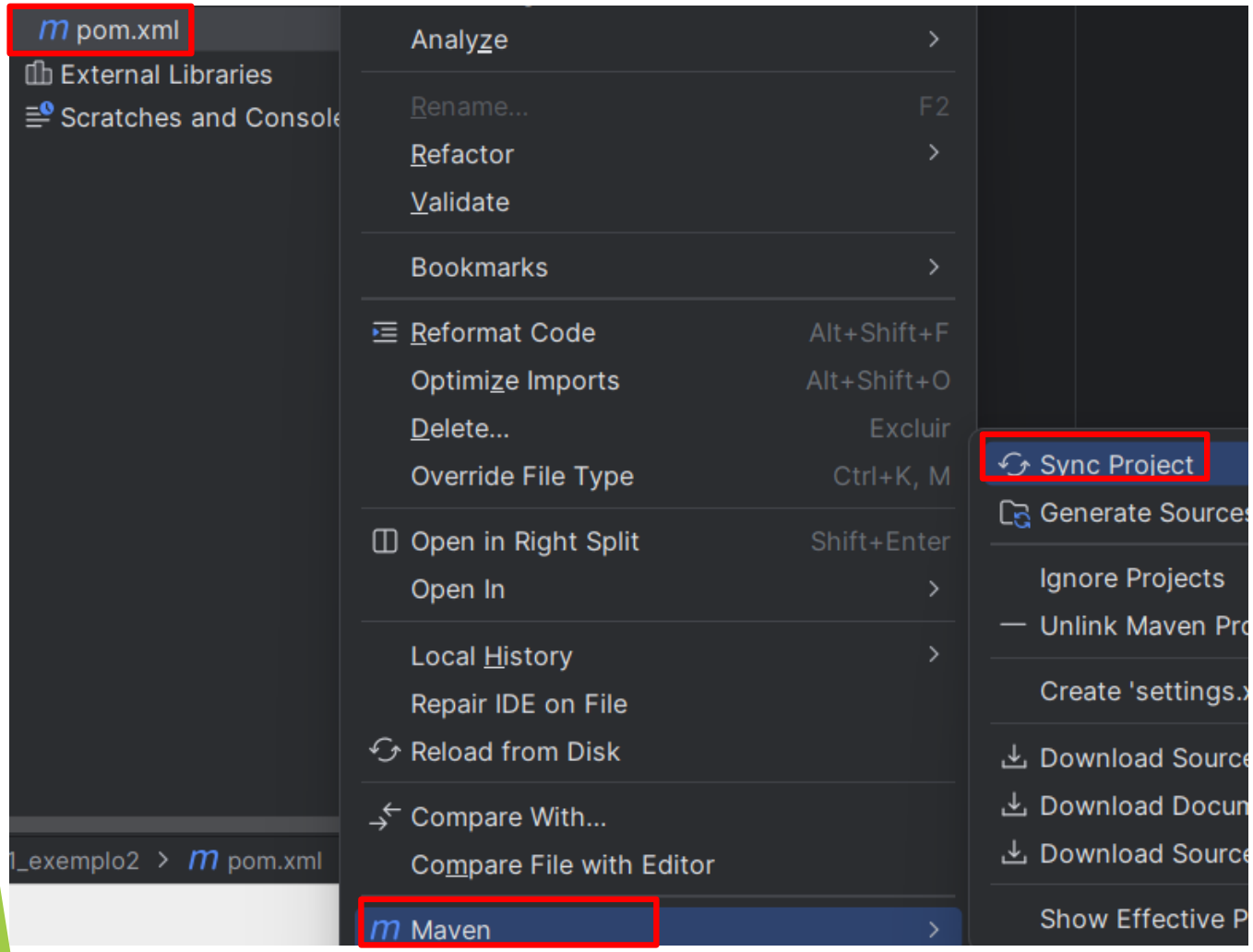
Depois crie uma classe dentro do controller , mesmo processo pressione botão direito , no botão controller , New->Java Class , coloque o nome da classe, com as Iniciais em Maiúsculo , IndexController.





PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

Para adicionar o maven no projeto , clique com botão direito no arquivo pom.xml , Maven->Sync Project



PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT



Defina a anotação @Controller, , clique em cima de @Controler e pressione ALT+ENTER , para definir que a classe será do tipo Controller e será criada automaticamente, e também automaticamente a importação será definida

```
package br.com.springboot.primeiroprojeto.controller;

import org.springframework.stereotype.Controller;

@Controller
public class IndexController {
}
```

PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

Crie um método abrirFormulario que terá com uma retorno uma String, na qual irá retornar uma View que iremos construir, e coloque uma anotação GetMapping , para passar os dados a página HTML.

```java
package br.com.spring.formularioProdutos.controller;

import org.springframework.stereotype.Controller;

import org.springframework.web.bind.annotation.GetMapping;


@Controller
public class IndexController {

    @GetMapping("/cadastro-produto")
    public String abrirFormulario() {

        return "cadastro"; // Abre a página html cadastro.html
    }
```

O @GetMapping é o que chamamos de "anotação composta". Ele já pré-configurado para aceitar apenas requisições do tipo GET. Neste será carregado no localohost:8080/cadastro-produto

Será chamado o arquivo do thymeleaf cadastro.html

PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

Crie um método salvar que terá com uma retorno uma String, na qual irá passar por parâmetro os dados do formulário html através da anotação RequestParam e o PostMapping Serve para **receber dados enviados** pelo formulárioao HTML.

```
package br.com.spring.formularioProdutos.controller;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestParam;

@Controller
public class IndexController {

    @GetMapping("/cadastro-produto")
    public String abrirFormulario() {
        return "cadastro"; // Abre a página html cadastro.html
    }

    @PostMapping("/salvar-produto")
    public String salvar(
            @RequestParam("nome") String nome,
            @RequestParam("preco") Double preco,
            @RequestParam("categoria") String categoria,
            Model model) {

        // Enviando os dados recebidos de volta para a pagina html
        model.addAttribute( attributeName: "n", nome);
        model.addAttribute( attributeName: "p", preco);
        model.addAttribute( attributeName: "c", categoria);
        model.addAttribute( attributeName: "sucesso", attributeValue: true);

        return "cadastro";
    }
```

O @PostMapping usada para indicar que um método do controller responderá a **requisições HTTP do tipo POST**. serve para **receber d enviados** pelo formulário. Depois será chamado a pagina salvar-proc mostrará os dados

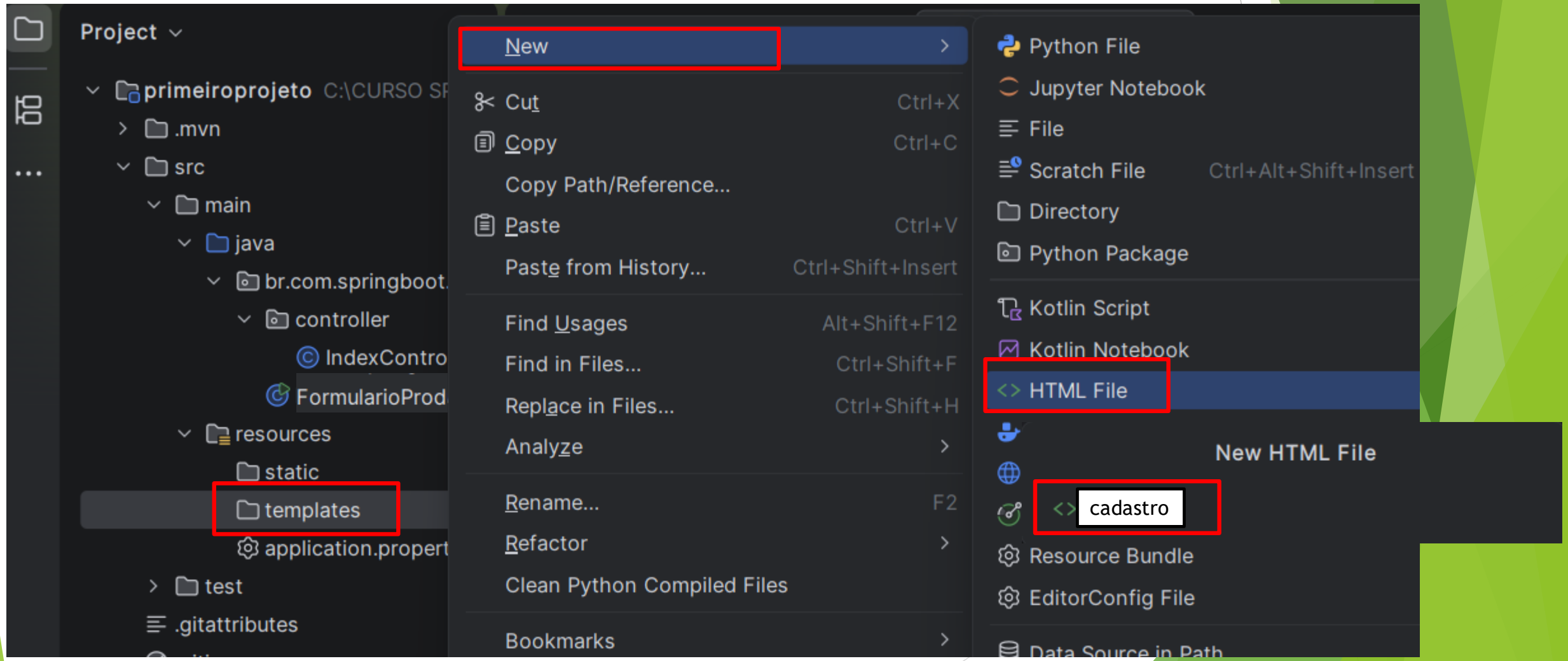
O **Model** transporta os dados do seu código Java (o Controller) para a sua página HTML (a View).

O método addAttribute , passa o nome do atributo e o valor , para o html



PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

No diretório resources, clique com botão direito em templates , selecione New->HTML File , crie um arquivo index.html



PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

No arquivo cadastro.html será criado um formulário para digitar os dados do produto e depois ao clicar em cadastrar será chamado o método salvar-produto

```
<!DOCTYPE html>
<html xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Sistema de Cadastro</title>
    <style>
        .formulario { margin-bottom: 15px; }
        label { display: block; font-weight: bold; }
        .resultado { background: #e7f3fe; padding: 15px; border-radius: 5px; margin-top: 20px; }
    </style>
</head>
<body>

<h2>Cadastrar Novo Produto</h2>

<form th:action="@{/salvar-produto}" method="post">
    <div class="formulario">
        <label>Nome do Produto:</label>
        <input type="text" name="nome" required placeholder="Ex: Monitor" />
    </div>

    <div class="formulario">
        <label>Preço:</label>
        <input type="number" step="0.01" name="preco" required />
    </div>

    <div class="formulario">
        <label>Categoria:</label>
        <select name="categoria">
            <option value="Eletrônicos">Eletrônicos</option>
            <option value="Móveis">Móveis</option>
            <option value="Vestuário">Vestuário</option>
        </select>
    </div>

    <button type="submit">Cadastrar Produto</button>
</form>
```

O thymeleaf deve ser declarado aqui para que os dados sejam recebidos do Controller

Ao preencher os dados chama o método salvar-produto via post

Botão que confirma o cadastro(submit)chama salvar-produto

PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT



No arquivo cadastro.html após o formulário , o thymeleaf verifica se o cadastro foi realizado com sucesso , e mostra as informações vinda do model , preenchidas no formulário

```
</form>

<div th:if="${sucesso}" class="resultado">
    <h3>✅ Produto Cadastrado com Sucesso!</h3>
    <p><strong>Nome:</strong> <span th:text="${n}"></span></p>
    <p><strong>Preço:</strong> R$ <span th:text="${p}"></span></p>
    <p><strong>Categoria:</strong> <span th:text="${c}"></span></p>
</div>

</body>
</html>
```

${sucesso} verifica se o cadastro foi realizado , retornará true no Controller, e mostrará os dados

Nomes dos atributos do Model

PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT

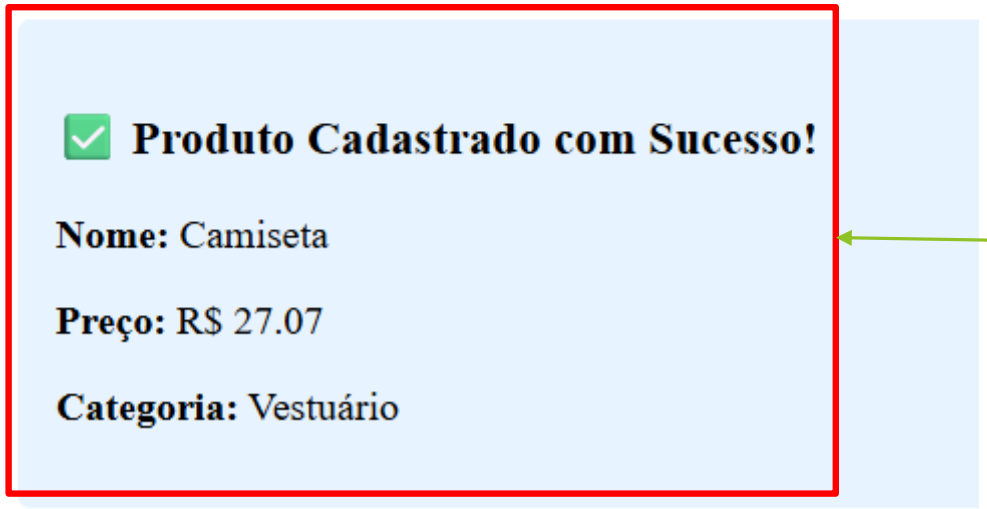
Para executar. Volte na classe Principal , clique no botão Run -> Run FormularioProd.....



PROJETO COM PASSAGEM DE PARÂMETROS E FORMULÁRIO COM SPRING BOOT



Cadastrar-produto



salvar-produto





EXERCÍCIOS Revisão com Request Param

Exercício 1

Exercício 1: CADASTRO DE BOLSAS DE ESTUDOS

Faça um formulário de cadastro de bolsas, utilize o request Param e mostre os dados

Bolsa
- CodigoCurso: integer -> utilize number - Nome_Curso : String -> utilize select - Carga_Horaria : String-> utilize number - Instituicao: String - Local : string

Exercício 2

Exercício 2: COMPRA DE ELETRODOMÉSTICOS

Faça um formulário de compra de eletrodomésticos, para mostrar os dados

Eletrodomestico
- codigo: integer -> utilize number - nome_eletrodomestico: String ->utilize select - valor : double -> utilize number - potência: double -> utilize number - cor : string

Exercício 3

Exercício 3: CADASTRO DE FORNECEDORES

Crie um projeto para cadastrar fornecedores , mostre os dados

Fornecedor
- codigo: integer -> utilize number - nomeFornecedor: String - nomeProduto: String ->utilize select - descricaoProduto: String

Exercício 4

Exercício 4: CADASTRO PASSAGEM DE ÔNIBUS

Crie formulário para cadastrar dados de uma passagem e mostre os dados

Passagem
-**codigoPassagem: integer** -> utilize number
-**nomePassageiro : String**
- RG: String
- localViagem: String
- data: String -> utilize date
- horário: String -> utilize number
- **numpoltrona: integer** -> utilize number